Developing an LLM-based Application for reef-keeping Diagnostics

1st William Merry
Artificial Intelligence Dept.
Worcester Polytechnic Institute
Nashville, TN, US
wmerry@wpi.edu

Github Repository
Production Application Link

***Abstract*—In-home reef-keeping is an ever-growing interest in the United States and globally. While these aquariums are amazing to look at, they are expensive investments requiring significant upkeep and an understanding of water chemistry. ReefXpert is an LLM-based chat application with a multi-stage data analysis pipeline to assist new practitioners in maintaining a healthy aquatic environment. This paper details the creation and performance evaluation of ReefXpert, including creation and training of a variational autoencoder to create synthetic data for the LLMs to analyze. Furthermore, a fault injection simulator is utilized to analyze the model's reaction to potential real-world system failures.**

***Index Terms*—Large-Language Model, Synthetic Data Generation, VAE, LLM Anomaly Diagnosis, reef-keeping, Aquarium Chemistry**

I. Introduction and Background

In-home coral reef systems are complex ecosystems that require rigorous testing and monitoring to ensure the safety and health of the inhabitants. However, not all who want to have a coral reef system are able to dedicate the required time or have the background to support the required knowledge. ReefXpert is an LLM-based chatbot and dashboard that leverages a multi-stage data pipeline to analyze reef system data. This tool was designed in hopes of making reef-keeping more accessible to all.

A. Market Analysis

The reef aquarium market is ever growing both in the United States and globally. As of 2022, there were approximately 2.5 million households keeping saltwater aquariums. In the same year, the industry was valued at $7,501.73 million, with an expected valuation of $12,674.01 million in 2031 and 6% growth CAGR over that period [1]. This data, paints a picture of a growing industry.

B. Coral Types and Costs

Coral kept in the home is typically categorized into the two broad groups with subgroups. These categories are based the presence or lack of presence of skeletal structure and are called stony coral and soft coral respectively. Stony corals are further categorized into large polyp stony (LPS) and small polyp stony (SPS). Care for each type differs greatly, making this knowledge important for reef-keepers.

Coral Type	Average Cost (USD)
Soft	$87.07
LPS	$67.14
SPS	$40.68
Overall	$61.64

TABLE I: Analysis of Coral Fragment Pricing by Category

Traditionally, online forums will direct new reef keepers towards soft coral or LPS coral. Soft coral and LPS coral require less light and water flow than their SPS counter parts. They are also much less sensitive to water parameter changes. As a result, keeping soft coral or LPS coral generally makes sense for a beginner. Less equipment investment is required and these coral types are typically much hardier.

An important consideration for anyone trying something new is cost. Reef keepers will often buy coral fragments (often called frags) to combat high colony prices. A frag is typically between 0.75" and 1.5" in diameter. To better quantify this, I compiled a small dataset of frag prices broken down by coral type from World Wide Corals [2]. World Wide Corals is a trusted vendor of coral based in the United States. See Table I for a breakdown of the average cost per frag by type. For further details on the data see appendix A.

Given the prices detailed in Table I, one can see the significant start up prices that may come with reef-keeping. In coral alone, a new reef keeper is likely to spend in excess of $500 for a small to medium sized tank. Add to this the cost of the equipment and additional livestock and a novice reef keeper can spend several thousand dollars on a fragile ecosystem, with no way to fully monitor it.

C. Aquarium Water Chemistry

Reef aquarium systems are very complex in home biochemistry labs that require rigorous testing and a depth of knowledge in marine chemistry. In a typical reef, calcium, alkalinity, magnesium, phosphate, nitrate, nitrite, ammonia,

salinity, water temperature, pH and ORP (redox potential) are all parameters that must be tested on a regular basis. In the below section, I provide details on some of these parameters, the criticality of the measurement, and relationships with other parameters. See Figure 1 for normal ranges for readings.

1) Calcium: Calcium is key for healthy stony coral growth. Stony coral will quickly deplete all calcium if it is not managed properly. For this reason, many reef keepers decide to dose calcium in small amounts to their aquariums. [3]

2) Alkalinity: Alkalinity is similarly critical for stony coral growth and health. Alkalinity is typically dependent on the water being used to mix saltwater with. This causes many reefers to dose alkalinity supplements to maintain proper values.

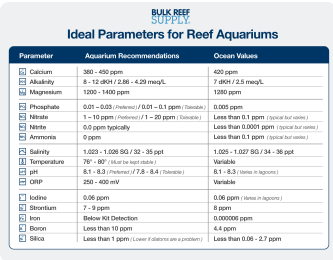
3) Phosphate: Phosphate is a organic nutrient that is good for reef tanks; however, excess phosphate can contribute to algae blooms. Phosphate often enters reef tanks through feeding and tap water. [3]

4) Ammonia: Ammonia is one of the most important most important parameters to monitor and test in a reef tank. Ammonia is created through biological decomposition of excess food and waste. An excess of ammonia in an aquarium can cause significant stress on the ecosystem and become toxic to the inhabitants. Ammonia levels are typically managed by the nitrogen cycle and water changes; however, they can spike dramatically with over-feeding or livestock death. [3]

5) Nitrite: The nitrogen cycle is key for aquarium health. During the first stage of the nitrogen cycle ammonia is converted to a slightly less toxic Nitrite (NO_2). Nitrite is toxic at levels above 5 mg/L. This conversion is completed by aerobic bacteria that oxidize ammonia to nitrite. [3]

6) Nitrate: Nitrate is the next parameter that is altered by the nitrogen cycle. Here, a group of aerobic bacteria oxidize nitrite into nitrate(NO_3). Nitrate is the least toxic of the three nitrogen containing parameters. In some cases, it can be a helpful nutrient for plants and algae. For fish it is often toxic after 40 mg/L. [3]

7) Salinity: Salinity is potentially one of the most important measures a reef keeper tracks. Salinity is a measure of the total dissolved salt in the water. Salinity, when out of balance can significantly impact the health of coral and fish in the aquarium.



BULK REEF SUPPLY

Ideal Parameters for Reef Aquariums

Parameter	Aquarium Recommendations	Ocean Values
Calcium	380 - 450 ppm	420 ppm
Alkalinity	8 - 12 dKH / 2.86 - 4.29 meq/L	7 dKH / 2.5 meq/L
Magnesium	1200 - 1400 ppm	1280 ppm
Phosphate	0.01 – 0.03 (*Preferred*) / 0.01 – 0.1 ppm (*Tolerable*)	0.005 ppm
Nitrate	1 – 10 ppm (*Preferred*) / 1 – 20 ppm (*Tolerable*)	Less than 0.1 ppm (*typical but varies*)
Nitrite	0.0 ppm typically	Less than 0.0001 ppm (*typical but varies*)
Ammonia	0 ppm	Less than 0.1 ppm (*typical but varies*)
Salinity	1.023 - 1.026 SG / 32 - 35 ppt	1.025 - 1.027 SG / 34 - 36 ppt
Temperature	76° - 80° (*Must be kept stable*)	Variable
pH	8.1 - 8.3 (*Preferred*) / 7.8 - 8.4 (*Tolerable*)	8.1 - 8.3 (*Varies in lagoons*)
ORP	250 - 400 mV	Variable
Iodine	0.06 ppm	0.06 ppm (*Varies in lagoons*)
Strontium	7 - 9 ppm	8 ppm
Iron	Below Kit Detection	0.000006 ppm
Boron	Less than 10 ppm	4.4 ppm
Silica	Less than 1 ppm (*Lower if diatoms are a problem*)	Less than 0.06 - 2.7 ppm

Fig. 1: Ideal Reef Aquarium Parameters as defined by Bulk Reef Supply [4]

II. Objective

The objective of this project is to create a dashboard and LLM chatbot to lower the barrier for entry to reef-keeping. This is accomplished by a web application which acts as a central dashboard for a saltwater reef aquarium. In this project, I created a synthetic dataset with the parameters detailed in Section I-C using a VAE trained on normal parametric ranges. The dashboard displays the generated data for the user to analyze in graph form. Additionally, the user is able to inject faults into the data to test the reasoning of the LLM, these faults are further detailed in Section V-D. Finally, there is a chatbot on the dashboard with a multi-step data analysis pipeline which ingests the data and provides the user diagnosis and maintenance advice for the upkeep of their aquarium.

A. Use Case Justification

Based on the information detailed in Section I and II, there is a need for a system like this and generative artificial intelligence is the perfect tool for the use case. Reef aquarium chemistry is a specialized subject that the average person does not have knowledge on. A text-based LLM integrated into a data visualization dashboard allows users to ask reef-specific questions and even take small samples of data to give the LLM as input. The data preprocessing allows for the LLM to easily identify trends and give relevant feedback to the user.

III. Selection of Generative AI Model

For this application, I chose Replicate as a hosting platform for the models. Replicate was selected due to the seamless API integration with the UI, the tool's thorough documentation, the simple pricing model, and the availability of the needed models for this project [5].

I have selected to use two versions of Meta's Llama. The first is Llama 3's 8 billion parameter version. This model was selected as the "light-weight" preprocessing model in the project. While 8 billion parameters is by no means small, this number served as a compromise between a smaller 1 or 2 billion parameter model and a much larger 20+ billion parameter version. This compromise was necessary, as the preprocessing model was implemented to return a set of filters to be applied to the raw water chemistry dataset quickly to not slow down overall inference time. For the task of taking a query and returning a simple JSON, a 1 billion parameter model would have likely sufficed; however, due to the niche focus of the project and lack of availability of a non-cold start 1 billion parameter model, a larger model was selected.

The model responsible for generating the visible responses to the user's query is Meta's Llama 4 Maverick. Llama 4 Maverick was selected as it is a multi-modal model allowing for input data of multiple types. While the input of the chatbot is purely text-based, a multi-modal LLM will generally perform better on structured data. This selection was also made with future project work in mind. In Section VII, I detail potential inclusion of image based analysis. Secondly, Llama 4 Maverick was selected for its mixture-of-experts architecture [6], the model has 17 billion active parameters and 128 experts,

totaling 400 billion parameters. Meta states that Llama 4 Maverick is "the best multimodal model in its class, beating GPT-4o and Gemini 2.0 Flash across a broad range of widely reported benchmarks, while achieving comparable results to the new DeepSeek v3 on reasoning and coding—at less than half the active parameters." [7] Additionally, the model has a low cost ratio of $0.95 per million output tokens and $0.25 per million input tokens, making it a good choice for both performance and resource efficiency.

IV. Project Definition and Use Case

The concept for this application is a question-answering system and diagnostic agent. When brainstorming for this idea, I had to create a list of the key functions that the application must complete.

A. Key Functions of the Application

1) Serve as a general aquarium question answering chatbot.
2) Perform data filtering and formatting for LLM Input.
3) Provide diagnostic suggestions on structured water chemistry data.
4) Visualize the data in a dashboard that allows user interaction.
5) Inject faults into the water parameter data at the user's request.
6) Create random faults and hide the issue from the user, allowing hypothesis testing.
7) Create a settings page on the dashboard, allowing the user to update the meta-parameters regarding the tank volume, equipment, age, lighting, and care information.

As detailed in Section III, the application will use two LLMs in the question answering and data analysis pipeline. The models are integrated into the application using a `replicate.stream()`. This was selected over `replicate.run()` as the stream command provides output tokens as they are being generated. This combined with a loop to continuously display the new updated text, allows the user to see the output tokens are they are being produced. This was selected mainly for user experience, to demonstrate a shorter perceived inference time.

V. Methodology

In the below section, I provide the details on the creation of the ReefXpert Dashboard and chatbot.

A. Resources Required

- Streamlit
- Replicate
- PyTorch
- Pandas
- Python Datetime and Time
- Python JSON
- Python Random
- Python Torchmetrics

For the hardware and cloud computing required, Replicate does not disclose the specific hardware they use for Llama 4 Maverick; however, it was designed to operate on a single NVIDIA H100 GPU. Replicate operates its models a few different GPU types, NVIDIA H100 is one of them [5]. Replicate does disclose that Llama 3 8b is run on NVIDIA H100 GPUs. For the case of this project it is a reasonable assumption that both models are run on NVIDIA H100 GPUs.

B. Synthetic Dataset Creation via VAE

To create the dataset that is fed into the dashboard for visualization and into the LLM for analysis, a variational autoencoder is leveraged. The VAE is created and trained in the `reefdatasetgen.py` module within the GitHub release linked at the top of this file.

1) Preprocessing: The input data into the model is comprised of 11 channels, one for each of the parameters defined in Section I-C. For each of these channels, a set of 50 random samples is generated, within the normal bounds of each chemical parameter. This set is purposely small in order to simulate a potential small data set in the future.

2) Model Architecture: The model follows a traditional VAE architecture with an input dimension of 11x50, a hidden layer dimension of 16 and a latent dimension of 32. This is further detailed in the below sections.

3) Loss Function: I implemented a standard loss function for VAEs. [8] This loss function is rooted in a balance between two parts, the Mean Squared Error loss, measuring reconstruction, and the Kullback-Leibler Divergence, measuring the regularity of the latent space. The loss function relies on a β value to balance the two terms. In my implementation, a low beta value of 0.25 was selected. This value was originally tuned to 0.05, but the latent space at this level was far too unstable. This created noisy graphs which affected the accuracy of the LLM.

$$Loss = MSELoss + \beta \times KLDivergence \tag{1}$$

4) VAE Limitations: The main limitations of using a VAE for this use-case is the lack of a reputable and open source dataset for this VAE to be trained on. Without this dataset, the VAE is trained on a random set of values that fall within a reasonable range for these parameters. Since these are random ranges, there is no correlation between the parameters for the VAE to learn. However, a VAE was still selected to serve as a place holder and allow for quick future work once a dataset can be collected and created. In Section VII I highlight the opportunity to collect this data over a short period of time to retrain the VAE.

5) Training process: The VAE was trained through a standard iterative training loop for 500 epochs. For each epoch, the dataset was partitioned into batches of 32 data points which were passed into the model and evaluated by the loss function as defined in Section V-B3. After, this the gradients are generated via back propagation and the model parameters are updated via the Adam optimizer with a learning rate of 0.001. See Figure 2 for a graph of loss over time.

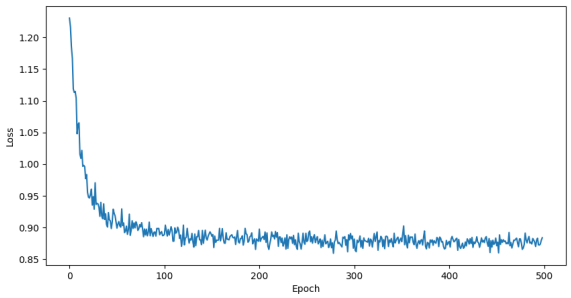


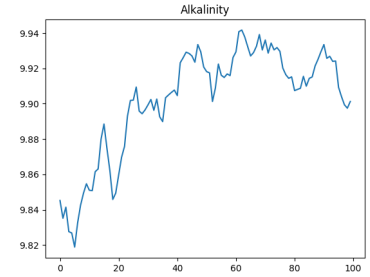
Fig. 2: Training Loss for Reef Data VAE

6) Random Walk: Next, I implemented a random walk function through the latent space. The function `generate_random_walk(model, T, latent_dim, step_size, device)` takes the model, T for time steps, the latent dimension, a step size and the device. In this case, the latent dim was set to 32 and the step size was set to 0.1. The random walk function first generates an array of zeros with size `(T, latent_dim)`. Then, the first row is selected and set to random values in the latent space. The remaining values are then stepped through and the step is added to the previous value before being multiplied by a random latent space value. Finally, the latent space vector is decoded to create an 11x100 array of parameters. See Figure 3 for example output plots.

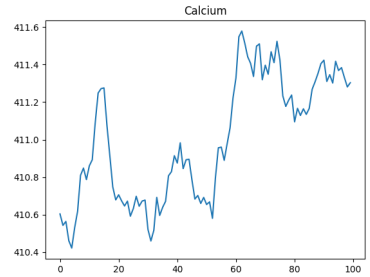
C. Data Pipeline Architecture

1) User Inputs: The data pipeline is built with a variety of user inputs. The first major user input is the query. This user query is fed directly into the Llama 3 8b Preprocessing model and the Llama 4 Diagnostic Model. The second type of user input is via the fault injection. As detailed later in Section V-D, the user is able to select from a group of pre-written faults on the fault injection page of the user interface. This directly injects faults into the synthetic data. Additionally, the data pipeline includes System Meta-Parameters, which are set to default values on application start; The user has the ability to edit them to their tank's meta values from a separate page as detailed in Section V-F3. The final input the pipeline takes is a true/false value via a toggle switch to turn on or off deep analysis mode. Depending on the state of deep analysis mode, the data is passed into the model differently. The main difference between using deep analysis mode and not is that with deep analysis mode toggled on, the Llama 3 8b model is engaged.

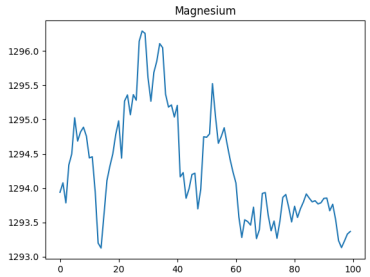
2) Deep Analysis Mode: This model acts as a preprocessesing step to generate a json file for the data filtering function to use. When deep analysis mode is enabled `get_data()` is called. Within get data is a Replicate API call to Llama 3 8b and a json decoder. The Llama model takes in the user query and returns a json in the following format: `{'time': 7, 'parameters': [[Calcium, Alkalinity, Magnesium]}`. This format is then read by a json decoder to grab the look back window and the parameters required. The data is then filtered to the n most recent values of the parameters required. This data is then passed into the Llama 4 Maverick Diagnostic model for a final output to return to the user. See Figure 4 for a system diagram when deep analysis mode is on.



(a) Example Plot of Alkalinity



(b) Example Plot of Calcium



(c) Example Plot of Magnesium

Fig. 3: Example Synthetic Data Plots

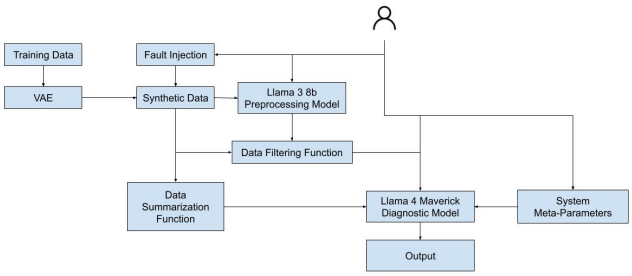


Fig. 4: System Diagram while using deep analysis Mode

3) Deep Analysis Mode Off: With deep analysis mode disabled, Llama 3 8b is never engaged. This was implemented

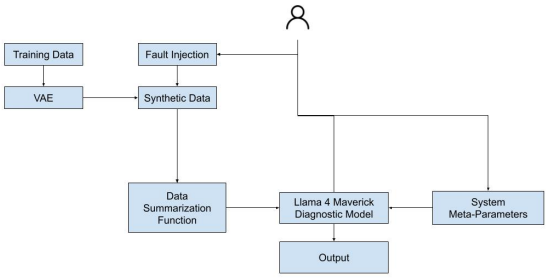


Fig. 5: System Diagram without using Deep Analysis Mode

in order to give the user optionality in the way their query was processed if they wanted a lower inference time. In this mode, the data is summarized before being passed into the diagnostic model, meaning the LLM still gets data context to use when generating a response. For example with deep analysis off, the diagnostic model would see something like the below in the system prompt:

```
Nitrate 50 Day History: From 4.44 to 4.34.
It decreased by 2.3 %
Nitrate 10 Day History: From 4.31 to 4.34.
It increased by 0.69 %
```

D. Fault Injection

The fault injection pipeline is kept intentionally simplistic without the use of any machine learning techniques. For each of the below faults, the `generate_steps()` function is run to generate new data. This function takes `parameter, max_steps_from_end, min_steps_from_end, increase, max_step, min_step` and a random time step is grabbed within bounds Next, the value at the time step is saved and the function steps through the remaining time steps and adds or subtracts a random step between the indicated min and max. Only one fault can be input at a time, and once another fault is selected the data is reset.

1) Random: This randomly selects a fault to inject into the data. The fault is by default hidden from the user so they can test the chat feature's ability to identify it.

2) Auto Top Off Failure: Reef tanks often have a system called an auto top off. This system is in place to automatically add freshwater to the system to combat evaporation. Without an auto top off, the salinity would rise due to a higher salt concentration in the remaining water. This fault is injected up to 30 days before the most recent reading and salinity steps up anywhere from 0 to 0.4 ppt daily.

3) Heater Failure On/Off: A common catastrophic failure in reef tanks is heater failure on or off. If the heater fails on, a rough heating element is in the tank and temperatures rise quickly. If the heater fails off, especially in colder climates, the temperature can drop quickly. This fault is injected up to 7 days before the most recent reading and water temperature steps up or down anywhere from 0 to 2 degrees daily.

4) Dosing Pump Failure: It is common for larger tanks to have dosing pumps to maintain parameters. With additional equipment comes another failure point. In this instance the assumption is the dosing pump will fail in the open position and continue to dose. This fault is injected into the alkalinity values up to 30 days before the most recent reading and steps up anywhere from 0 to 0.8 dKH daily.

5) Filtration Issue: Filtration issues are common in reef-keeping. The nitrogen cycle as outlined in I-C, is upheld by proper filtration. If the filtrion fails, the nitrogen cycle may collapse. This fault is injected into the values up to 21 days before the most recent reading and ammonia, nitrite and nitrate would increase by steps between 0 and 0.1 ppm. ORP would also drop between 0 and 0.2 mV daily.

6) Flow Issue: Flow issues cause similar symptoms as filtration issues, albeit less significant. This fault is injected into the values up to 21 days before the most recent reading and ammonia, nitrite and nitrate would increase by steps between 0 and 0.08 ppm. The pH would also drop between 0 and 0.1 daily.

7) Livestock Death: Livestock death would significantly affect the nitrogen cycle, for this reason, the faults injected match those of flow issues however the ammonia, nitrite and nitrate may step as much as 0.2 ppm.

8) Refugium Light Failure: This is a minor fault that would cause an increase in nitrate and phosphate. This fault may be injected up to 30 days before the most recent data point and will step up by between 0 and 0.1 ppm for each parameter daily.

9) Protein Skimmer Failure: Protein skimmers are responsible for removing excess proteins from the water column. If a protein skimmer were to fail, phosphate and nitrate would increase while ORP would decrease. The fault can be injected up to 21 days before the last data point and nitrate can increase by 0.1 ppm, phosphate by 0.01 ppm and ORP may decrease by 0.2 mV.

E. Deployment Strategy

For this application, I have chosen to use streamlit cloud due to the ease of use and extensive documentation. Streamlit has built-in modules for chat functions and graphing. With these being the largest pieces of this use case, it was a logical choice for deployment. Streamlit cloud's ability to connect directly into Github and sync to repository updates makes it a good choice for future iterations [9].

F. UI implementation

As mentioned discussed in above sections, Streamlit was selected for the model's user interface. The subsections below detail the three pages within the application.

1) Dashboard UI: This serves as the main page for the application. In the center of the page are the graphs for the 11 water parameters so the user can see their parameters over time. On the left-hand side of this screen is a sidebar with the navigation at the top and the LLM chat interface. The chat interface has a normal chat window which streams context back in real-time from Replicate. This provides the user with a Chat-GPT like feel. This section also has a reset button to

reset chat history and the toggle switch to enable deep analysis mode.

2) Fault Injection UI: This page gives the user the ability to select one of 9 faults as outlined in Section V-D. When selecting the fault, it initially stays hidden. In the top right of the screen is a 'Show Fault' button to display the selection. This was implemented to keep a random fault hidden from the user so they are able to test the bot. In the top left is a reset data button if the user wants to remove all injected faults from the data.

3) Settings and Meta-Parameters UI: This page serves as a form for users to input meta-parameters about their tank. It is purely a form submission with no other features to the UI. See the questions in the form below:

- What is the volume of the total system? (gallons)
- What is the age of your aquarium? (years)
- What is the estimated flow/water turnover rate in your aquarium? (GPH)
- Are you using a refugium?
- Are you using a protein skimmer?
- Are you using a carbon reactor?
- What type of lighting do you use?
- How many hours are your lights on per day?
- Per week, what percentage water change do you do? If you perform water changes on a non-weekly basis, please normalize the number to weekly.

VI. Model Evaluation & Discussion

For the evaluation of this model and system architecture, I have decided to measure two key metrics the latency in receiving an answer to a query and the accuracy of the answer. These are measured by inference time and ROUGE (Recall-Oriented Understudy for Gisting Evaluation) score respectively. In the sections below, I detail the performance of the standard model and deep analysis model across these two metrics.

A. Inference Time

Inference time is a key metric to analyze for this use case due to the nature of the the two modes proposed and implemented in Section V. In Table II, the average of inference time metrics a detailed for a set of predictions made on queries with deep analysis mode on and off. For these queries a fault was injected and the LLM was prompted to diagnose the fault. The measured being analyzed in the tables within this section are: Input Tokens, Output Tokens, Total Inference Time (seconds), Time to First Token (seconds), Tokens/Second. The data for this metric was collected directly from Replicate on the Replicate Dashboard where API call data is provided.

Table II shows the average inference time with deep analysis mode on and off across a set of faults. Each model was fed the exact same set of input queries. The inference time metrics per fault are detailed in Table V for standard mode and Table VI for deep analysis mode. With deep analysis mode off, detailed data segments are not passed into the model. This is represented in the tokens in column of Table II. The models' system prompts are otherwise identical which is represented in the similar output token count.

The most important of the metrics shown in Table II are Total Time and TTFT and as a result these were prioritized in the system designs. With deep analysis mode off, the model fully returned a prediction in an average of 3.02 seconds, however this time jumped to 4.07 seconds with deep analysis mode on. Similarly TTFT increases significantly from 560 milliseconds to 990. For more details on the ROUGE-1 scores see the Tables in Appendix B.

Mode	Tokens In	Tokens Out	Total Time	TTFT	Tok/sec
Standard	1041	121	3.02	0.56	69.69
Deep Analysis	1819.67	136	4.07	0.99	33.71

TABLE II: Average Inference Time

B. Rouge Score

ROUGE Score or Recall-Oriented Understudy for Gisting Evaluation is a set of natural language processing metrics used to evaluate the quality of generated text. These metrics are the F1 score, precision and recall. This is the key other half of the data to be able to evaluate the strength of the approach. For measuring the ROUGE score, I used ROUGE-1 from torchmetrics. ROUGE-1 was selected as the analysis required word-by-word analysis rather than over multiple words.

Before the predictions were passed into ROUGE-1, they were filtered to only include key words. The key words included the water parameter names from the dataset and all potential fault names.

The ROUGE-1 function takes two inputs, the prediction and the ground truth. In my implementation, I created ground truths for the each of the potential fault injections. These ground truths were simply the fault name and parameter names. For example, the ground truth for a query regarding data with a Filter Issue Fault was "filter issue orp ammonia nitrate nitrite".

Table III shows that the recall between the two methods are identical, suggesting both models are successfully able to identify the correct fault the same percentage of the time. However, the precision and F1 scores are where they differ. Using the application with deep analysis mode off surprisingly provided higher precision, meaning the model provided less fault and parameter keywords in its response. The F1 score also showed strength in the standard mode over the deep analysis mode. For more details on the ROUGE-1 scores see the Tables in Appendix C.

Mode	F1	Precision	Recall
Standard	0.52	0.37	0.94
Deep Analysis	0.42	0.28	0.94

TABLE III: Filtered ROUGE-1 Scores

C. Discussion

Based on the data presented in Section VI-A and Section VI-B, I would suggest leveraging the standard mode over the deep analysis mode when utilizng this tool. Looking at the inference time metrics, the standard architecture was on average 34.7% quicker than the deep analysis mode. Additionally, the TTFT measure for the standard architecture was 76% faster than the deep analysis mode.

Total inference time and time to first token are important measures for user experience of an application as too much latency in a tool and a user may not opt to use it. Considering the better time performance of the standard architecture and the similar rouge metrics between the two architectures as presented in Section VI-B, in future work on this project I do not believe I would integrate the deep analysis mode in the current state. It does not provide enough benefit to outweigh its disadvantages in inference time.

VII. Conclusions and Future Work

In this project, I proposed and implemented a novel application to detect faults in an under-explored use case for machine learning and generative artificial intelligence. Reef aquarium keeping is a large and growing interest in North America and around the globe. This paper proposed ReefXpert, an application aimed at lowering the barrier for entry to this expensive and difficult to learn passion project.

A. Dataset Creation

In future iterations of this project, I intend to create a dataset with real world reef data in order to for the VAE to better learn the relationships between the parameters. For instance, the VAE would better learn the nitrogen cycle and the data would represent the cyclical nature between parameters.

B. Image Analysis

Since the Llama 4 Maverick is a multi-modal model, there is an opportunity to take images as an input. In future iterations, I intend to create an image analysis pipeline to allow the user to pass images with their queries.

C. Hardware Creation

Finally, in future iterations, there is an opportunity to implement hardware with this application. There are some examples of open-source or commercial automated reef testing systems. This would involve creating a piece of testing hardware to extend this project and allow the dashboard to automatically update. There is also an opportunity to include preemptive fault detection and suggest preventative maintenance with no user input.

References

[1] A. Sharma, "Reef Aquarium Market Size, Share and Forecast to 2031," Straits Research, Tech. Rep., Jul. 2023. [Online]. Available: https://straitsresearch.com/report/reef-aquarium-market

[2] "Buy Corals, Supplies, Fish and Inverts for your Reef Tank." [Online]. Available: https://worldwidecorals.com/

[3] R. J. Goldstein, *Marine Reef Aquarium Handbook*, 1st ed. Hauppague, New York: Barron's, Jan. 1997.

[4] "Reef Tank Water Parameters," Oct. 2025. [Online]. Available: https://www.bulkreefsupply.com/content/post/reef-tank-parameters

[5] "Documentation – Replicate." [Online]. Available: https://replicate.com/docs

[6] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V. Le, G. E. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," *CoRR*, vol. abs/1701.06538, 2017. [Online]. Available: http://arxiv.org/abs/1701.06538

[7] "The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation," Apr. 2025. [Online]. Available: https://ai.meta.com/blog/llama-4-multimodal-intelligence/

[8] D. P. Kingma and M. Welling, "An introduction to variational autoencoders," *CoRR*, vol. abs/1906.02691, 2019. [Online]. Available: http://arxiv.org/abs/1906.02691

[9] "Streamlit Docs." [Online]. Available: https://docs.streamlit.io/

Appendix A
Coral Pricing Data

Sale Price	LPS Coral	SPS Coral	Soft Coral
$10	1	1	0
$15	1	0	0
$19	26	15	1
$29	22	23	1
$35	0	0	1
$39	26	17	2
$49	16	7	1
$59	15	8	2
$69	5	2	0
$79	14	2	5
$89	4	1	2
$99	7	1	9
$119	7	0	1
$129	3	0	1
$139	3	0	0
$149	11	0	1
$159	2	0	0
$199	2	0	0
$249	1	1	1
$299	1	0	0
$399	1	0	0

TABLE IV: Detailed View of Number of Corals Listed at each price by category

Appendix B
Detailed Tables for Inference Time

Fault	Tokens In	Tokens Out	Total Time	TTFT	Tok/sec
Filter Issue	1033	117	1.44	0.55	132.63
ATO Failure	1046	116	3.94	0.55	34.37
Dosing Pump	1045	130	3.68	0.58	42.06

TABLE V: Inference Time with Deep Analysis Off

Fault	Tokens In	Tokens Out	Total Time	TTFT	Tok/sec
Filter Issue	1990	134	4.37	0.97	30.66
ATO Failure	1404	139	3.57	0.99	38.94
Dosing Pump	2065	135	4.28	1.01	31.54

TABLE VI: Inference Time with Deep Analysis On

Appendix C
Detailed Tables for ROUGE Score

Fault	F1	Precision	Recall
Filter Issue	0.5263	0.3846	0.8333
ATO Failure	0.3000	0.1765	1.0000
Dosing Pump	0.4444	0.2857	1.0000

TABLE VII: Rouge-1 Scores with Deep Analysis On

Fault	F1	Precision	Recall
Filter Issue	0.6667	0.5556	0.8333
ATO Failure	0.4615	0.3333	1.0000
Dosing Pump	0.4211	0.2667	1.0000

TABLE VIII: Rouge-1 Scores with Deep Analysis Off